The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Abstract

Contents

1 Introduction

This chapter some basics on blockchain concepts useful to understand this project.

1.1 What is a blockchain?

A blockchain is an immutable ledger that enables the recording of transactions in a P2P network, providing a single source of truth. The first one was invented by Satoshi Nakamoto in 2008 with the release of the Bitcoin whitepaper [5]. It developed a fully peer-to-peer (P2P) network where users can exchange a currency (bitcoin) without third party authorities with the power of rewriting the history of transactions.

A blockchain can be described as a system made of multiple components, the foundamentals are:

- a P2P network of nodes;
- a consensus algorithm;
- a chain of blocks, containing the verified transactions.

There are many variants of blockchains, an important difference to consider to classify them is who can access the network. In a permissionless blockchain (like Ethereum or Bitcoin) anybody can access, anyone can participate in the consensus; permissioned blockchains, on the other hand, restrict the access using a whitelist for trusted peers.

Cryptography primitives

Blockchains rely heavily on cryptographic techniques to ensure security and trust. The main cryptographic tools used are:

Hash functions are one-way mathematical functions that take any input and produce a fixed-size output (called a hash or digest). They are deterministic, meaning the same input always produces the same output. However, it is computationally impossible to reverse the process or find two different inputs that produce the same hash. Bitcoin and Ethereum use SHA-256 and Keccak-256 hash functions respectively.

Digital signatures allow users to prove ownership of their assets and authorize transactions. Each user has a private key (which must be kept secret) and a public key (which can be shared). To send a transaction, a

user signs it with their private key. Anyone can verify the signature using the user's public key, proving that the transaction was authorized by the owner of that private key. Most blockchains use elliptic curve cryptography for digital signatures.

How transactions work

When a user wants to make a transaction on a blockchain, they create a message containing the details (such as the recipient and amount) and sign it with their private key. This signed transaction is then broadcast to the P2P network.

Nodes in the network collect these transactions and group them into blocks. Before a block can be added to the blockchain, it must be validated through the consensus mechanism. Once consensus is reached, the block is added to the chain, and all nodes update their copy of the ledger. This process ensures that all participants have the same view of transaction history.

The chain structure is maintained by including the hash of the previous block in each new block. This creates a tamper-proof link: if someone tries to modify an old transaction, it would change that block's hash, breaking the link to all subsequent blocks. This is why blockchains are described as immutable.

Consensus mechanisms

The consensus mechanism is how the network agrees on which transactions are valid and the order in which they should be recorded. This is a critical challenge in distributed systems, where nodes might be unreliable or malicious [2].

Proof of Work

Bitcoin introduced **Proof of Work** (PoW), where nodes (called miners) compete to solve complex mathematical puzzles. Specifically, miners must find a nonce value that, when combined with the block data and hashed, produces a hash below a certain target threshold. This requires repeated trial and error, making it computationally intensive. The difficulty of the puzzle is adjusted periodically to maintain a consistent block time—approximately 10 minutes per block in Bitcoin, while Ethereum (before its transition to Proof of Stake) targeted roughly 12-15 seconds per block.

The first miner to solve the puzzle gets to add the next block and receives a reward in the form of newly minted cryptocurrency plus transaction fees. This process requires significant computational power and electricity, making it expensive to attack the network. An attacker would need to control more than 50% of the network's total computational power to reliably manipulate the blockchain.

PoW provides strong security through this computational cost, making attacks economically infeasible. It has a proven track record, securing Bitcoin since 2009, and remains permissionless—anyone with computational resources can participate. However, PoW suffers from significant drawbacks. The computational work requires massive amounts of electricity, leading to environmental concerns. Mining also tends to concentrate in areas with cheap electricity, creating centralization risks. Furthermore, the need for specialized hardware (ASICs) creates barriers to entry for individual participants.

Proof of Stake

Proof of Stake (PoS) was developed as an alternative to address PoW's high energy consumption. Instead of computational work, PoS selects validators based on the amount of cryptocurrency they have locked up as collateral (their "stake"). Validators are chosen to propose and validate blocks based on factors such as the size of their stake and how long they've been staking. When selected, a validator creates a new block and other validators attest to its validity. Validators who act dishonestly risk losing their stake through a process called "slashing," creating an economic incentive for honest behavior.

The fundamental differences between PoW and PoS reflect different security models. PoS requires capital (staked tokens) rather than computational power, making it dramatically more energy efficient—consuming only a fraction of the energy compared to PoW. The attack model also differs: in PoS, attacking the network requires acquiring and risking a significant stake, whereas PoW requires controlling computational resources. Block production is deterministic in PoS, with validators selected according to protocol rules, rather than the competitive race to solve puzzles characteristic of PoW.

PoS offers several advantages, particularly its minimal computational requirements, which eliminate the need for specialized hardware and lower barriers to entry. The economic security model creates strong disincentives for attacks, as malicious behavior results in financial penalties. However, PoS faces its own challenges. The "nothing at stake" problem suggests that validators could theoretically validate multiple competing chains without cost. There are also concerns about wealth concentration, as those with more tokens have more influence over the network. Finally, PoS is newer and less battle-tested than PoW, having less operational history in securing high-value networks.

1.2 Ethereum and smart contracts

While Bitcoin was designed primarily as a digital currency, Ethereum extended the blockchain concept to support general-purpose computation [2]. Launched in 2015 by Vitalik Buterin and others, Ethereum introduced **smart contracts**:

programs deployed to the blockchain that execute when invoked by a transaction. A smart contract's code is stored on-chain and cannot be altered after deployment. It does not run by itself—it only executes when a user or another contract sends a transaction that calls one of its functions. Every node in the network then runs the same code with the same inputs and reaches the same result, which is what makes the execution trustless.

Ethereum consensus mechanism

Ethereum initially used Proof of Work with the Ethash algorithm, which was designed to be ASIC-resistant and more memory-intensive than Bitcoin's SHA-256. However, the Ethereum community recognized the limitations of PoW, particularly its energy consumption and scalability constraints.

In September 2022, Ethereum underwent one of the most significant upgrades in blockchain history, known as **The Merge**. This event transitioned Ethereum from Proof of Work to Proof of Stake, implementing a consensus mechanism called **Gasper**, which combines the Casper FFG (Friendly Finality Gadget) and LMD GHOST (Latest Message Driven Greediest Heaviest Observed SubTree) protocols.

In Ethereum's PoS system, validators must stake 32 ETH to participate in block production and validation. These validators are randomly selected to propose blocks in assigned time slots every 12 seconds, while other validators attest to the validity of proposed blocks. Validators earn rewards for honest participation and face penalties (slashing) for malicious behavior. Finality is achieved through a checkpoint system, where blocks become irreversible after being finalized. This transition reduced Ethereum's energy consumption by approximately 99.95% while maintaining security and enabling future scalability improvements.

Smart contracts and the EVM

A smart contract is a program written in a high-level language (such as Solidity or Vyper) and deployed to the blockchain. Once deployed, the contract's code cannot be changed. Correct execution is ensured by redundancy: when a transaction calls a contract, every node in the network independently executes the same bytecode with the same inputs using a deterministic virtual machine. If a node produces a different result, the other nodes reject it through the consensus mechanism. No single node can alter the outcome, because the majority of the network must agree on the final state. This guarantee—immutable code plus redundant deterministic execution—is what enables trustless decentralized applications (DApps) that operate without central control.

Smart contracts are executed by the **Ethereum Virtual Machine** (EVM), a quasi-Turing-complete computational environment that runs on every node in the network. The EVM is a stack-based virtual machine with a word size of

256 bits, designed to execute bytecode compiled from high-level smart contract languages. When a transaction calls a smart contract, every node executes the same code and updates their state accordingly, ensuring that all nodes maintain consensus on the state of all accounts and contracts.

The EVM maintains several types of state: **account state** (balances and nonces for externally owned accounts), **contract state** (storage, code, and account balance for smart contracts), and **transaction state** (temporary data during transaction execution).

Opcodes

At the lowest level, the EVM executes **opcodes**: simple operations that form the instruction set of the virtual machine. Smart contract bytecode consists of a sequence of these opcodes, each performing a specific operation. The EVM instruction set includes arithmetic operations (ADD, SUB, MUL, DIV, MOD), comparison operations (LT, GT, EQ, ISZERO), bitwise operations (AND, OR, XOR, NOT, BYTE), and stack operations (PUSH, POP, DUP, SWAP). Memory and storage are manipulated through dedicated opcodes—MLOAD and MSTORE for temporary memory, while SLOAD and SSTORE handle persistent contract storage. Control flow is managed by jump instructions (JUMP, JUMPI, JUMPDEST), and the EVM provides access to blockchain context through opcodes like ADDRESS, BALANCE, CALLER, CALLVALUE, TIMESTAMP, and NUMBER. Finally, execution opcodes like CALL, DELEGATECALL, STATICCALL, CREATE, and SELFDESTRUCT enable contract interaction and deployment.

Each opcode has an associated gas cost that reflects its computational complexity and resource usage. For example, simple arithmetic operations cost 3 gas, while writing to storage (SSTORE) costs significantly more (20,000 gas for setting a new value) because it requires updating the permanent blockchain state.

Gas mechanism

To prevent infinite loops and abuse, Ethereum uses a concept called **gas**. Gas serves multiple critical purposes in the Ethereum ecosystem. First, it provides **resource metering**, ensuring every computation on the EVM costs a certain amount of gas proportional to the computational resources required. This metering prevents denial-of-service attacks by requiring payment for computation, making it economically infeasible for attackers to overwhelm the network with expensive operations. Gas also serves as **validator compensation**—users pay for the gas their transactions consume, compensating validators for processing transactions and storing state. Finally, gas creates a **priority mechanism** where users can offer higher gas prices to incentivize validators to include their transactions faster.

The gas system operates through a straightforward model. Each operation has a fixed **gas cost** measured in gas units. Users specify a **gas limit** (the maximum gas they're willing to consume) and a **gas price** or max fee (the amount of ETH they'll pay per unit of gas). The total transaction cost equals gas used multiplied by gas price. If a transaction runs out of gas before completing, it reverts, but the gas spent up to that point is still consumed as compensation for the computational work performed. Unused gas is refunded to the sender.

Since the London hard fork (EIP-1559), Ethereum uses a more sophisticated fee market with a **base fee** that is automatically adjusted based on network congestion and burned (removed from circulation), plus an optional **priority fee** (tip) that goes to validators. This creates more predictable transaction fees and introduces a deflationary mechanism for ETH.

Different types of operations have vastly different gas costs reflecting their resource requirements. Simple arithmetic operations cost only 3-5 gas, while SHA3 hashing requires 30 gas plus 6 gas per word. Storage operations are particularly expensive: reading from storage (SLOAD) costs 2,100 gas, while writing to storage (SSTORE) costs 20,000 gas for setting a new value or 5,000 gas for modifying an existing one. Contract creation requires 32,000 gas plus the deployment cost, and every transaction has a base cost of 21,000 gas. This pricing structure incentivizes developers to write efficient code and minimize on-chain storage usage, as storage operations are among the most expensive.

Decentralized applications

Smart contracts enable a wide range of applications beyond simple payments. Some examples include:

- **Decentralized Finance (DeFi)**: Financial services like lending, borrowing, and trading without traditional intermediaries.
- **Non-Fungible Tokens (NFTs)**: Unique digital assets that represent ownership of items like art, collectibles, or real-world assets.
- **Decentralized Autonomous Organizations (DAOs)**: Organizations governed by smart contracts where decisions are made through voting by token holders.
- **Supply chain tracking**: Recording the movement of goods through a supply chain with transparency and immutability.

The key advantage of these applications is that they inherit the blockchain's properties: they are transparent, censorship-resistant, and don't require trust in a central authority. However, they also face challenges such as scalability, high transaction costs, and the irreversibility of bugs in smart contract code.

2 Crypto and the Real World

The previous chapter introduced blockchain technology and Ethereum from a technical perspective. This chapter bridges the gap between the technical infrastructure and the practical question: how does Ethereum interact with the real world? It explains how people acquire tokens, what kinds of assets exist, why anyone would hold crypto instead of traditional savings, and why overcollateralized borrowing—which may seem paradoxical at first—is actually one of the most powerful tools in decentralized finance.

2.1 Exchanges: From Euros to Tokens

To use any blockchain-based service, a user first needs to acquire tokens. The most common way to do this is through a **cryptocurrency exchange**, which acts as a bridge between traditional currencies (fiat) and digital assets.

Centralized exchanges

Centralized exchanges (CEXs) such as Coinbase, Binance, and Kraken work similarly to online brokerages. A user creates an account, completes identity verification (known as Know Your Customer or KYC), and links a bank account or credit card. They can then buy cryptocurrencies directly with euros, dollars, or other fiat currencies.

The process is straightforward:

1. Register and verify identity on the exchange.
2. Deposit fiat money via bank transfer or card payment.
3. Place a buy order for the desired cryptocurrency (e.g., ETH, USDC).
4. Optionally, withdraw the purchased tokens to a personal wallet.

Centralized exchanges hold custody of the user's funds while they remain on the platform. This makes the experience familiar—similar to using a bank—but introduces a trust requirement: users depend on the exchange to safeguard their assets. Exchange failures, such as the collapse of FTX in 2022, have demonstrated the risks of this custodial model [4].

Decentralized exchanges

Decentralized exchanges (DEXs) such as Uniswap operate entirely on the blockchain through smart contracts. Instead of a central order book, DEXs use **liquidity pools**: smart contracts where users deposit token pairs. Traders swap one token for another directly against these pools, with prices determined by an automated market maker (AMM) algorithm.

DEXs do not require identity verification and never hold custody of user funds. However, they cannot accept fiat currency directly—a user must already hold some cryptocurrency to use a DEX. In practice, most newcomers start with a centralized exchange to convert fiat to crypto, and later use decentralized exchanges for token-to-token swaps.

2.2 Types of Crypto Assets

The term "crypto" encompasses more than just currencies. Several distinct types of assets exist on Ethereum:

- **Native tokens**: ETH is Ethereum's native currency. It is used to pay transaction fees (gas), participate in staking, and serves as the base unit of value across the ecosystem.
- **Stablecoins**: Tokens pegged to a traditional currency, typically the US dollar. Examples include USDC (backed by dollar reserves held by Circle) and DAI (backed by overcollateralized crypto deposits). Stablecoins allow users to hold dollar-denominated value on the blockchain without exposure to crypto price volatility.
- **Governance tokens**: Tokens that grant voting rights in a protocol's decision-making process. For example, AAVE token holders can vote on protocol upgrades, risk parameters, and treasury allocations.
- **Wrapped tokens**: Representations of assets from other blockchains. Wrapped Bitcoin (WBTC) is an ERC-20 token on Ethereum backed 1:1 by Bitcoin held in custody, allowing Bitcoin holders to participate in Ethereum's DeFi ecosystem.
- **Non-Fungible Tokens (NFTs)**: Unique tokens representing ownership of a specific item—digital art, game items, domain names, or real-world asset certificates.
- **Liquidity Provider (LP) tokens**: Tokens received when depositing into a DEX liquidity pool. They represent the user's share of the pool and can themselves be used in other DeFi protocols.

These assets are not isolated. DeFi protocols allow them to be combined: a user might deposit ETH to earn interest, use the interest-bearing aETH as collateral to borrow USDC, and then provide that USDC to a liquidity pool. This composability is what makes the ecosystem powerful—and what the previous chapter described as "money legos."

2.3 Earning Yield: Staking vs. Bank Savings

One of the most tangible reasons people move assets into crypto is the potential for higher returns compared to traditional savings.

Traditional bank savings

In traditional finance, a savings account in a European bank typically offers an annual interest rate between 0.5% and 3%, depending on the institution, country, and whether the deposit is locked for a fixed term. This interest comes from the bank lending deposited funds to borrowers at higher rates, keeping the spread as profit.

For context, as of early 2025, typical European savings rates are:

- Demand deposits (instant access): 0.5%–1.5% APY
- Fixed-term deposits (1 year lock): 2%–3% APY

These rates are guaranteed by the bank and protected by deposit insurance schemes (up to €100,000 per depositor per bank in the EU), making them extremely low-risk.

Staking ETH

Ethereum's Proof of Stake consensus mechanism (described in Chapter 1) creates a native way to earn yield. Validators who stake 32 ETH to secure the network receive rewards funded by transaction fees and newly issued ETH. As of early 2025, **ETH staking yields approximately 3%–4% APY**.

Users who do not have 32 ETH can participate through **liquid staking** services like Lido. Users deposit any amount of ETH and receive a liquid staking token (stETH) that represents their staked position. This token accrues staking rewards over time and can be freely traded, used as collateral, or deposited into other DeFi protocols. This means the user earns staking yield while retaining liquidity—something impossible with a bank's fixed-term deposit.

Supplying to lending protocols

Beyond staking, users can earn yield by supplying assets to lending protocols like AAVE. When a user deposits USDC into AAVE, they earn interest paid by

borrowers. These rates fluctuate with supply and demand: when many users want to borrow USDC, the interest rate rises; when few do, it falls. Typical supply rates for stablecoins on AAVE range from 2% to 8% APY depending on market conditions.

The key differences between these options and a bank account are:

	Bank savings	**ETH staking**	**AAVE supply**
Typical APY	0.5%–3%	3%–4%	2%–8%
Access	Instant or locked	Liquid (via Lido)	Instant
Risk	Deposit insurance	Smart contract risk	Smart contract risk
Custody	Bank holds funds	Self-custody	Self-custody
Requirements	KYC, bank account	ETH holdings	Any supported token

Higher yields come with different risks. There is no government deposit insurance in DeFi. Smart contract bugs, oracle failures, or governance attacks could result in loss of funds. However, protocols like AAVE have been running since 2020, securing billions of dollars, and have been extensively audited. The risk profile is different from a bank account—not necessarily higher or lower, but different in nature.

2.4 Why Borrow Overcollateralized?

The most counterintuitive aspect of DeFi lending is overcollateralization: to borrow $1,000, a user must lock up more than $1,000 in collateral. If one already has the assets, why borrow at all?

This question has several concrete answers:

Accessing liquidity without selling

Consider a user who holds 10 ETH, currently worth $30,000, and believes the price will rise. They need $5,000 to pay for a car repair. Without DeFi, they would have to sell some ETH—triggering a capital gains tax event and losing future upside if the price increases.

With a protocol like AAVE, they can instead deposit their 10 ETH as collateral and borrow $5,000 in USDC. They pay the car repair, keep their ETH exposure, and repay the loan later—possibly after ETH has appreciated. The borrowing cost (interest) is typically 2%–5% APY, which may be far less than the potential upside of holding ETH.

Tax efficiency

In many jurisdictions, selling cryptocurrency is a taxable event that triggers capital gains tax. Borrowing against crypto holdings is **not a sale**, and therefore typically does not trigger a tax obligation. This allows users to access the

economic value of their holdings without the tax consequences of liquidating them.

Leveraged positions

Advanced users use borrowing to create leveraged positions. For example:

1. Deposit 10 ETH as collateral.
2. Borrow USDC against the ETH.
3. Use the borrowed USDC to buy more ETH.
4. Deposit the additional ETH as more collateral.

This cycle amplifies exposure to ETH price movements. If ETH rises, the gains are multiplied. If ETH falls, the losses are also amplified and the position risks liquidation. This is conceptually similar to margin trading in traditional finance.

Yield farming

Users can borrow assets at one rate and supply them elsewhere at a higher rate, capturing the spread. For example, a user might borrow USDC at 3% from AAVE and supply it to another protocol offering 6%, netting a 3% profit on the borrowed amount. The collateral continues earning its own yield simultaneously.

A practical example

To make this concrete, consider the following scenario:

Alice holds 10 ETH (worth $30,000) and wants to earn yield on her position while accessing some liquidity. She:

1. Deposits 10 ETH into AAVE, earning approximately 1% supply APY ($300/year).
2. Borrows 10,000 USDC against her ETH collateral at 3% variable rate ($300/year in interest).
3. Stakes the 10,000 USDC in a stablecoin yield vault earning 5% APY ($500/year).
4. Net result: Alice earns $300 + $500 – $300 = $500/year in yield, while maintaining full exposure to ETH price appreciation and having accessed $10,000 in spendable value.

Compare this to a traditional bank: Alice could sell her ETH for $30,000, deposit it in a savings account at 2% APY ($600/year), but she would lose all exposure to ETH price movements and trigger a taxable event on any gains.

This example illustrates why DeFi borrowing exists despite overcollateralization. The collateral is not "locked away uselessly"—it continues earning yield, maintains price exposure, and the borrowed funds create additional earning opportunities. The net position can be significantly more capital-efficient than simply holding assets idle.

3 Background

This chapter introduces the key technologies and concepts that form the foundation of this project: decentralized finance, the AAVE lending protocol, and modular smart accounts.

3.1 Decentralized Finance

Decentralized Finance, commonly known as DeFi, refers to financial services built on blockchain technology that operate without traditional intermediaries like banks or brokers. Instead of relying on centralized institutions, DeFi applications use smart contracts to automate financial operations, making them transparent, permissionless, and accessible to anyone with an internet connection.

Core concepts

Traditional finance relies on trusted intermediaries to facilitate transactions. When you deposit money in a bank, the bank holds your funds and decides how to use them. When you take out a loan, the bank evaluates your creditworthiness and sets the terms. DeFi replaces these intermediaries with code: smart contracts that execute automatically according to predefined rules.

The key principles of DeFi include:

- **Permissionless access**: Anyone can use DeFi services without needing approval from a central authority. There are no credit checks, identity verification, or geographic restrictions.
- **Transparency**: All transactions and smart contract code are publicly visible on the blockchain. Users can verify exactly how protocols work and audit their security.
- **Composability**: DeFi protocols can interact with each other, allowing developers to combine different services like building blocks. This is often called "money legos."
- **Self-custody**: Users maintain control of their own assets through their private keys, rather than trusting a third party to hold their funds.

Token standards

Tokens are digital assets that exist on a blockchain. On Ethereum, the most common token standard is **ERC-20**, which defines a common interface for fungible tokens. Fungible means that each token is identical and interchangeable with any other token of the same type, just like how one euro is equivalent to any other euro.

The ERC-20 standard specifies functions that all compliant tokens must implement:

- `balanceOf(address)`: Returns how many tokens an address owns.
- `transfer(address, amount)`: Sends tokens to another address.
- `approve(address, amount)`: Allows another address (typically a smart contract) to spend tokens on your behalf.
- `transferFrom(from, to, amount)`: Transfers tokens from one address to another, used by approved spenders.

This standardization is crucial for DeFi because it allows protocols to work with any ERC-20 token without needing custom code for each one. A lending protocol can accept any ERC-20 token as collateral, and a decentralized exchange can trade any pair of ERC-20 tokens.

Lending and borrowing

One of the most fundamental DeFi services is lending and borrowing. In traditional finance, if you want to earn interest on your savings, you deposit money in a bank. If you need a loan, you apply to the bank and provide collateral or proof of income.

DeFi lending protocols work differently. Users who want to earn interest deposit their tokens into a **liquidity pool**, a smart contract that holds funds from many different users. These pooled funds are then available for other users to borrow.

Borrowers must provide **collateral** to take out a loan. Because DeFi is permissionless and pseudonymous, there is no way to enforce repayment through legal means. Instead, borrowers must lock up assets worth more than what they borrow—for example, depositing ETH or stablecoins like USDC as collateral to borrow a different token. If the value of their collateral drops below a certain threshold (the **liquidation threshold**), anyone can repay part of the loan and claim the collateral at a discount. This mechanism ensures that lenders are always protected, even if borrowers default.

Interest rates in DeFi lending are typically determined algorithmically based on supply and demand. When there is high demand for borrowing a particular asset, interest rates increase to attract more lenders. When demand is low, rates decrease.

3.2 AAVE Protocol

AAVE is one of the largest and most widely used DeFi lending protocols [1]. Originally launched as ETHLend in 2017, it was rebranded to AAVE (Finnish for "ghost") in 2020 with the release of AAVE V2. The protocol has since evolved to AAVE V3, which introduced new features for capital efficiency and risk management.

How AAVE works

AAVE operates as a system of liquidity pools, with one pool for each supported asset. Users interact with the protocol through a central `Pool` contract that manages all lending and borrowing operations.

Supplying assets: When users supply assets to AAVE, they deposit tokens into the protocol and receive **aTokens** in return. These aTokens represent the user's share of the liquidity pool and automatically accrue interest over time. For example, if you supply 100 DAI, you receive 100 aDAI. As borrowers pay interest, the balance of aDAI in your wallet increases. When you want to withdraw, you redeem your aTokens for the underlying asset plus accumulated interest.

Borrowing assets: To borrow from AAVE, users must first supply collateral. The amount they can borrow depends on the **Loan-to-Value (LTV)** ratio of their collateral. For example, if an asset has an LTV of 80%, a user who supplies $1000 worth of that asset can borrow up to $800. AAVE supports two types of interest rates: **stable rates** that remain relatively constant, and **variable rates** that fluctuate based on market conditions.

Liquidations: If a borrower's collateral value drops and their position becomes undercollateralized (meaning their debt exceeds the allowed threshold), liquidators can repay part of the debt and receive the collateral at a discount. This mechanism protects lenders and maintains the protocol's solvency.

AAVE V3 features

AAVE V3 introduced several improvements over previous versions:

- **Efficiency Mode (E-Mode)**: Allows higher borrowing power when supplying and borrowing correlated assets, such as different stablecoins.
- **Isolation Mode**: Limits exposure to riskier assets by capping how much can be borrowed against them.
- **Portal**: Enables seamless transfer of liquidity between AAVE deployments on different networks.
- **Gas optimizations**: Reduced transaction costs through more efficient smart contract design.

AAVE smart contracts

The AAVE protocol consists of several key smart contracts:

- `Pool`: The main entry point for users. Handles supply, borrow, repay, and withdraw operations.
- `PoolAddressesProvider`: A registry that stores the addresses of all protocol contracts, making it easier to interact with the protocol.
- `AToken`: The interest-bearing token that represents supplied assets.
- `VariableDebtToken` and `StableDebtToken`: Tokens that represent borrowed amounts with variable or stable interest rates.
- `Oracle`: Provides price feeds for assets, used to calculate collateral values and determine liquidations.

To interact with AAVE programmatically, external contracts call functions on the `Pool` contract. The most commonly used functions include:

- `supply(asset, amount, onBehalfOf, referralCode)`: Deposits tokens into the protocol.
- `borrow(asset, amount, interestRateMode, referralCode, onBehalfOf)`: Borrows tokens against supplied collateral.
- `repay(asset, amount, interestRateMode, onBehalfOf)`: Repays borrowed tokens.
- `withdraw(asset, amount, to)`: Withdraws supplied tokens from the protocol.

3.3 Account Abstraction

Traditional Ethereum accounts, known as Externally Owned Accounts (EOAs), are controlled by private keys. Every transaction must be signed by the private key, and the account has limited functionality: it can only send transactions and hold assets. This creates several problems:

- **Key management**: Losing a private key means losing access to all funds permanently.
- **No recovery options**: There is no way to recover an EOA if the key is lost or compromised.
- **Gas payment**: Users must always have ETH to pay for gas, even if they only want to interact with other tokens.

- **Limited automation**: EOAs cannot execute transactions automatically or set complex conditions.

Account Abstraction addresses these limitations by replacing EOAs with smart contract accounts, often called **Smart Accounts**. These accounts are smart contracts that can implement custom logic for authentication, recovery, and transaction execution.

ERC-4337

ERC-4337 is a standard that brings account abstraction to Ethereum without requiring changes to the protocol itself [3]. It introduces a new transaction flow:

1. Users create **UserOperations** instead of regular transactions. These describe what action the user wants to perform.
2. UserOperations are sent to a separate **mempool** (a waiting area for pending operations) rather than the regular transaction mempool.
3. **Bundlers** collect UserOperations and submit them to the blockchain as regular transactions.
4. A singleton contract called the **EntryPoint** processes the bundled operations and calls the target smart accounts.
5. Each smart account validates the operation according to its own rules and executes the requested action.

This architecture enables features that were previously impossible:

- **Custom authentication**: Smart accounts can use any validation logic, not just ECDSA signatures. This enables multi-signature wallets, social recovery, biometric authentication, and more.
- **Sponsored transactions**: A third party (called a **Paymaster**) can pay for gas on behalf of users, enabling gasless transactions.
- **Batched transactions**: Multiple operations can be combined into a single transaction, saving gas and improving user experience.

ERC-7579: Modular Smart Accounts

While ERC-4337 defines how smart accounts interact with the network, it does not specify how accounts should be structured internally. Different wallet providers implemented their own architectures, leading to fragmentation in the ecosystem.

ERC-7579 addresses this by defining a standard interface for **modular smart accounts** [6]. Instead of building monolithic smart accounts with all features hardcoded, ERC-7579 enables accounts to install and uninstall modules that add specific functionality.

The standard defines four types of modules:

- **Validators**: Modules that verify whether an operation is authorized. They can implement different authentication schemes like ECDSA signatures, multi-sig, passkeys, or social recovery.
- **Executors**: Modules that can trigger actions on behalf of the account. They enable automation, scheduled transactions, and integration with external protocols.
- **Hooks**: Modules that run before and/or after execution. They can enforce policies, implement spending limits, or add additional security checks.
- **Fallback handlers**: Modules that handle calls to functions not defined on the account itself, enabling extensibility.

Each module type has a specific interface it must implement. For example, all modules must implement:

- `onInstall(bytes data)`: Called when the module is installed on an account.
- `onUninstall(bytes data)`: Called when the module is removed from an account.
- `isModuleType(uint256 typeID)`: Returns whether the module is of a specific type.

Executor modules

Executor modules are particularly relevant for DeFi automation. They allow external contracts or accounts to trigger actions through the smart account, following predefined rules. Common use cases include:

- **Automated DeFi strategies**: Executors can automatically rebalance positions, harvest yields, or manage liquidity based on market conditions.
- **Scheduled transactions**: Recurring payments or periodic operations can be triggered by executors.
- **Protocol integrations**: Executors can provide simplified interfaces for interacting with complex DeFi protocols.

An executor module can call the smart account's `execute` function to perform actions like token transfers, contract calls, or multi-step operations. The modular architecture ensures that these actions are performed with the account's permissions while maintaining security through the account's validation logic.

This combination of account abstraction and modular architecture creates a powerful foundation for building sophisticated DeFi applications that can interact with protocols like AAVE in automated and user-friendly ways.

4 Analysis and Design

5 Implementation

This chapter describes the implementation of the AAVE executor module, covering the supply, withdraw, borrow, and repay flows. I explain how these operations work in the AAVE protocol, how my executor module wraps them for use with ERC-7579 smart accounts, and how I test the implementation against a mainnet fork. For the borrow flow, I also introduce key DeFi concepts such as Loan-to-Value ratio, health factor, and interest accrual. The repay flow closes the borrowing lifecycle by allowing a smart account to pay back debt and restore its position's health.

5.1 Supply Flow

AAVE supply mechanism

When a user wants to supply assets to AAVE, the protocol requires two steps. First, the user must approve the Pool contract to spend their tokens. This is a standard ERC-20 approval that grants the Pool permission to transfer tokens on behalf of the user. Second, the user calls the `supply` function on the Pool contract.

The Pool contract's supply function has the following signature:

```
function supply(
    address asset,
    uint256 amount,
    address onBehalfOf,
    uint16 referralCode
) external;
```

Listing 5.1: AAVE Pool supply function

The `asset` parameter specifies which token to supply, `amount` is how much to deposit, `onBehalfOf` determines who receives the aTokens (allowing one address to supply on behalf of another), and `referralCode` is used for referral tracking (typically set to 0).

When supply succeeds, AAVE transfers the specified amount of tokens from the caller to the Pool and mints an equivalent amount of aTokens to the `onBehalfOf` address. These aTokens represent the user's share of the liquidity pool and automatically accrue interest over time.

Executor module supply implementation

My executor module wraps this two-step process into a single execution flow. When the smart account calls the `execute` function with the SUPPLY action, the module performs both the approval and the supply call on behalf of the account.

```
if (action == SmartAAVE.ActionAAVE.SUPPLY) {
    _execute(asset0, 0, abi.encodeCall(IERC20.approve, (pool, amount)));
    _execute(
        pool,
        0,
        abi.encodeCall(IPool.supply, (asset0, amount, onBehalfOf, 0))
    );
}
```

Listing 5.2: Supply implementation in the executor

The `_execute` function is inherited from `ERC7579ExecutorBase` and allows the executor module to make calls through the smart account. Each `_execute` call causes the smart account itself to perform the specified action. This means the approval is granted from the smart account's address to the Pool, and the supply call originates from the smart account.

The first `_execute` call approves the Pool contract to spend the specified amount of `asset0` (configured during module installation). The second call invokes `pool.supply()`, transferring tokens from the smart account to AAVE and minting aTokens back to the `onBehalfOf` address.

5.2 Withdraw Flow

AAVE withdraw mechanism

Withdrawing from AAVE is simpler than supplying because no approval is needed. The user holds aTokens, and the Pool contract can burn them directly when processing a withdrawal. The withdraw function signature is:

```
function withdraw(
    address asset,
    uint256 amount,
    address to
) external;
```

Listing 5.3: AAVE Pool withdraw function

The `asset` parameter specifies which underlying token to withdraw, `amount` is how much to withdraw (or `type(uint256).max` to withdraw everything), and `to` is the address that receives the withdrawn tokens. When withdraw

executes, AAVE burns the corresponding aTokens from the caller and transfers the underlying asset to the `to` address.

Because aTokens accrue interest continuously, the amount withdrawn can exceed the original amount supplied. The interest earned depends on the utilization rate of the pool and the time elapsed since the supply.

Executor module withdraw implementation

The withdraw implementation in my executor is straightforward since only one call is needed:

```
if (action == SmartAAVE.ActionAAVE.WITHDRAW) {
    _execute(
        pool,
        0,
        abi.encodeCall(IPool.withdraw, (asset0, amount, onBehalfOf))
    );
}
```

Listing 5.4: Withdraw implementation in the executor

The `_execute` call makes the smart account invoke `pool.withdraw()`, which burns the account's aTokens and sends the underlying tokens to the specified recipient. In my implementation, I use `onBehalfOf` as the recipient, allowing the smart account to withdraw funds either to itself or to another address.

5.3 Borrow Flow

Borrowing is one of the core features of AAVE that enables users to leverage their supplied assets. Unlike simply supplying tokens to earn yield, borrowing introduces additional concepts and risks that users must understand.

Key concepts

Before diving into the implementation, I explain three fundamental concepts that govern borrowing in AAVE: Loan-to-Value ratio, health factor, and interest accrual.

Loan-to-Value (LTV)

The Loan-to-Value ratio represents the maximum amount a user can borrow against their collateral, expressed as a percentage. For example, if USDC has an LTV of 77%, a user who deposits $10,000 worth of USDC can borrow up to $7,700 worth of other assets.

The formula for LTV is:

$$\text{LTV} = \frac{\text{Borrowed Value}}{\text{Collateral Value}} \times 100 \tag{5.1}$$

Each asset in AAVE has its own maximum LTV parameter, which reflects the protocol's assessment of that asset's risk profile. Stablecoins like USDC typically have higher LTV ratios (around 77%) because their prices are relatively stable, while more volatile assets have lower LTV ratios.

Health factor

The health factor is AAVE's primary metric for measuring how safe a borrowing position is from liquidation. It is calculated as:

$$\text{Health Factor} = \frac{\text{Collateral Value} \times \text{Liquidation Threshold}}{\text{Total Debt}} \tag{5.2}$$

The liquidation threshold is a percentage (slightly higher than the maximum LTV) at which a position becomes eligible for liquidation. For USDC, this is typically around 80%.

The health factor indicates the position's safety:

- **Health Factor** > 1: The position is safe and cannot be liquidated
- **Health Factor** $= 1$: The position is at the liquidation threshold
- **Health Factor** < 1: The position is underwater and can be liquidated by anyone

Several factors can cause the health factor to decrease: the collateral asset's price dropping, the borrowed asset's price rising, or interest accruing on the debt over time.

Interest accrual

When users borrow from AAVE, they pay interest on their debt. AAVE uses a variable interest rate model where the rate depends on the pool's utilization—the higher the percentage of the pool that is borrowed, the higher the interest rate. This mechanism incentivizes liquidity provision when demand for borrowing is high.

Interest compounds continuously, meaning the debt grows over time even if the user takes no action. AAVE tracks this efficiently using a "borrow index" that increases over time. A user's actual debt is calculated as their scaled debt balance multiplied by the current borrow index.

This continuous interest accrual means that even if asset prices remain stable, a borrower's health factor will slowly decrease over time as their debt grows relative to their collateral.

AAVE borrow mechanism

To borrow from AAVE, a user must first have supplied collateral. The borrow function signature is:

```
function borrow(
    address asset,
    uint256 amount,
    uint256 interestRateMode,
    uint16 referralCode,
    address onBehalfOf
) external;
```

Listing 5.5: AAVE Pool borrow function

The `asset` parameter specifies which token to borrow, `amount` is how much to borrow, `interestRateMode` selects between stable (1) or variable (2) interest rates, `referralCode` is for referral tracking, and `onBehalfOf` specifies who receives the borrowed tokens and assumes the debt.

When borrow executes, AAVE checks that the user has sufficient collateral to support the new debt while maintaining a healthy position. If the check passes, AAVE mints variable debt tokens to track the user's obligation and transfers the borrowed asset to the specified address.

Executor module borrow implementation

My executor module implements the borrow action to allow smart accounts to borrow assets from AAVE:

```
if (action == SmartAAVE.ActionAAVE.BORROW) {
    _execute(
        pool,
        0,
        abi.encodeCall(IPool.borrow, (asset, amount, 2, 0, onBehalfOf))
    );
}
```

Listing 5.6: Borrow implementation in the executor

The implementation uses variable interest rate mode (2) as this is the most common choice in practice. The `_execute` call makes the smart account invoke `pool.borrow()`, which transfers the borrowed tokens to the `onBehalfOf` address after verifying the account has sufficient collateral.

To query a position's health status, I also expose the Pool's `getUserAccountData` function through the interface:

```
function getUserAccountData(address user)
    external
    view
```

```
returns (
    uint256 totalCollateralBase,
    uint256 totalDebtBase,
    uint256 availableBorrowsBase,
    uint256 currentLiquidationThreshold,
    uint256 ltv,
    uint256 healthFactor
);
```

Listing 5.7: Getting user account data

This function returns all relevant metrics for monitoring a borrowing position, including the current LTV, liquidation threshold, and health factor.

5.4 Repay Flow

Repaying closes the borrowing lifecycle. Once a user has borrowed assets from AAVE, they accumulate debt that grows over time due to interest accrual. Repaying reduces or eliminates that debt, restoring the position's health factor and eventually freeing the collateral for withdrawal.

Key concepts

Debt tokens

When a user borrows from AAVE, the protocol mints *variable debt tokens* to the borrower's address. These tokens represent the outstanding obligation and automatically rebase upward as interest accrues. Repaying burns these debt tokens proportionally to the amount repaid. A full repayment burns all debt tokens and removes the borrowing position entirely.

Partial versus full repayment

AAVE supports both partial and full repayment. A partial repayment reduces the outstanding debt and raises the health factor without closing the position. A full repayment eliminates all debt for the specified asset.

Because interest accrues continuously, the exact amount owed at repayment time can be slightly larger than the amount originally borrowed. To avoid leaving dust debt, AAVE accepts `type(uint256).max` as the amount, which instructs the protocol to repay the entire outstanding balance regardless of the exact figure. In my implementation I pass the explicit amount rather than the max sentinel, which is appropriate for the educational context of this module.

Repayment and collateral

Repaying does not automatically return collateral. After repaying the debt, the user must separately call withdraw to reclaim the supplied assets. The two operations are intentionally decoupled in AAVE, which gives users flexibility to maintain a partial collateral position even after repaying.

AAVE repay mechanism

Like supply, repaying requires a prior ERC-20 approval because the Pool contract must pull the repayment tokens from the caller. The repay function signature is:

```
function repay(
    address asset,
    uint256 amount,
    uint256 interestRateMode,
    address onBehalfOf
) external returns (uint256);
```

Listing 5.8: AAVE Pool repay function

The `asset` parameter specifies which borrowed token to repay, `amount` is how much debt to clear, `interestRateMode` must match the mode used when borrowing (variable rate is 2), and `onBehalfOf` identifies whose debt is being repaid. The function returns the actual amount repaid, which may differ from the requested amount when using the max sentinel.

Executor module repay implementation

My executor module implements the repay action symmetrically to supply: it first approves the Pool to pull the tokens, then calls repay.

```
if (action == SmartAAVE.ActionAAVE.REPAY) {
    _execute(asset, 0, abi.encodeCall(IERC20.approve, (pool, amount)));
    _execute(
        pool,
        0,
        abi.encodeCall(IPool.repay, (asset, amount, 2, onBehalfOf))
    );
}
```

Listing 5.9: Repay implementation in the executor

The first `_execute` call approves the Pool to spend the repayment amount from the smart account. The second call invokes `pool.repay()`, which burns the corresponding debt tokens and transfers the repayment amount from the smart account back to the pool. The interest rate mode is fixed at 2 (variable), consistent with how the borrow action was implemented.

5.5 Testing on a Mainnet Fork

Testing DeFi integrations requires interacting with real protocol state. I use Foundry's forking capabilities to run tests against actual AAVE contracts deployed on Ethereum mainnet.

Fork setup

The test suite connects to a mainnet fork using the real AAVE V3 Pool address and token addresses:

```
contract ExecutorTemplateTest is RhinestoneModuleKit, Test {
    AccountInstance internal instance;
    ExecutorTemplate internal executor;
    address pool = 0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2;
    address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;

    function setUp() public {
        init();
        executor = new ExecutorTemplate();
        instance = makeAccountInstance("ExecutorTemplate");
        vm.deal(address(instance.account), 10 ether);
        instance.installModule({
            moduleTypeId: MODULE_TYPE_EXECUTOR,
            module: address(executor),
            data: abi.encode(pool, WETH, USDC)
        });
    }
}
```

Listing 5.10: Test contract setup

The `RhinestoneModuleKit` base contract provides helpers for creating smart accounts and installing modules. I create a new smart account instance and install my executor module, passing the pool address and supported assets as initialization data.

Testing supply

To test the supply flow, I use Foundry's `deal` cheatcode to mint WETH directly to the smart account, then execute a supply operation:

```
function testExecSupply() public {
    deal(WETH, address(instance.account), 10 ether);
    supply();
    assertEq(IERC20(WETH).balanceOf(address(instance.account)), 0);
```

```
}

function supply() public {
    bytes memory supplyData = abi.encode(
        WETH,
        10 ether,
        address(instance.account),
        0,
        0  // ActionAAVE.SUPPLY
    );
    instance.exec({
        target: address(executor),
        value: 0,
        callData: abi.encodeWithSelector(
            ExecutorTemplate.execute.selector,
            supplyData
        )
    });
}
```

Listing 5.11: Supply test

The test verifies that after supplying, the smart account's WETH balance is zero, confirming that all tokens were transferred to AAVE. The account now holds aWETH tokens instead, representing its position in the lending pool.

Testing withdraw

The withdraw test builds on the supply test by first supplying tokens, then withdrawing them:

```
function testExecWithdrawSuccess() public {
    deal(WETH, address(instance.account), 10 ether);
    supply();
    bytes memory withdrawData = abi.encode(
        WETH,
        10 ether,
        address(instance.account),
        0,
        1  // ActionAAVE.WITHDRAW
    );
    instance.exec({
        target: address(executor),
        value: 0,
        callData: abi.encodeWithSelector(
            ExecutorTemplate.execute.selector,
```

```
            withdrawData
        )
    });
    assertGt(IERC20(WETH).balanceOf(address(instance.account)), 10);
}
```

Listing 5.12: Withdraw test

The assertion `assertGt(..., 10)` verifies that the withdrawn amount exceeds the original supply. This confirms that the AAVE integration is working correctly and that interest accrues on supplied assets. Even in the short time between supply and withdraw in the test, the forked mainnet state includes accumulated interest from real protocol activity.

Testing borrow with fuzz testing

The borrow functionality requires more comprehensive testing due to the additional complexity of collateralization, health factors, and interest accrual. I use Foundry's fuzz testing capabilities to test the borrow flow across a range of input values, with each test configured to run at least 10 iterations. All tests use USDC as collateral and borrow WETH.

I implemented five fuzz tests to verify different aspects of the borrow functionality:

1. **Basic borrow success**: Verifies that borrowing works correctly for various amounts. The test supplies 10,000 USDC as collateral, borrows a fuzzed amount of WETH (between 0.001 and 1 ETH), and confirms the borrowed tokens were received. The `bound` function constrains the fuzz input to amounts that should succeed given the collateral.

2. **Health factor verification**: Confirms that after borrowing within safe limits, the health factor remains above 1 (represented as `1e18` in AAVE's 18-decimal precision). This ensures positions created by the executor are safe from immediate liquidation.

3. **Interest accrual**: Uses Foundry's `vm.warp` cheatcode to advance blockchain time by one year, then verifies that the debt has increased. This demonstrates that borrowing has ongoing costs—users must account for interest when managing their positions.

4. **Liquidation conditions**: Simulates a collateral price crash by using `vm.mockCall` to override the Chainlink oracle's `latestAnswer()` function, making it return near-zero for USDC. With the collateral worthless, the health factor drops below 1, confirming that price volatility can make positions liquidatable.

5. **High LTV positions**: Tests larger positions with $50,000 USDC collateral and borrowing 5-10 ETH. This exercises higher-value scenarios while verifying the position maintains a safe health factor and records a non-zero LTV.

These tests collectively validate that the borrow implementation correctly interacts with AAVE's lending protocol and that the key risk metrics (health factor, LTV, debt accrual) behave as expected.

Testing repay with fuzz testing

The repay tests build on the borrow setup and verify that repaying has the expected effect on the borrowing position. Like the borrow tests, they use USDC as collateral and WETH as the borrowed asset.

```
function repay(uint256 amount) internal {
    bytes memory data = abi.encode(
        WETH,
        amount,
        address(instance.account),
        0,
        3  // ActionAAVE.REPAY
    );
    instance.exec({
        target: address(executor),
        value: 0,
        callData: abi.encodeWithSelector(
            ExecutorTemplate.execute.selector,
            data
        )
    });
}
```

Listing 5.13: Repay helper function

I implemented two fuzz tests to verify different aspects of the repay functionality:

1. **Debt reduction**: Verifies that repaying reduces the outstanding debt. The test supplies 10,000 USDC as collateral, borrows a fuzzed amount of WETH, records the total debt via `getUserAccountData`, repays the same amount, and asserts that the debt after repayment is strictly less than before. This confirms that the approval and repay calls are correctly routed through the smart account.

   ```
   function testFuzz_RepayReducesDebt(uint256 borrowAmt) public {
       uint256 collateral = 10_000 * 10 ** USDC_DECIMALS;
   ```

```
        borrowAmt = bound(borrowAmt, 0.001 ether, 1 ether);
        deal(USDC, address(instance.account), collateral);
        supplyCollateral(collateral);
        borrow(borrowAmt);
        (, uint256 debtBefore,,,,) = IPool(pool)
            .getUserAccountData(address(instance.account));
        repay(borrowAmt);
        (, uint256 debtAfter,,,,) = IPool(pool)
            .getUserAccountData(address(instance.account));
        assertLt(debtAfter, debtBefore);
    }
```

Listing 5.14: Repay debt reduction test

2. **Health factor restoration**: Verifies that repaying can bring a liquidatable position back to safety. The test borrows enough to create a healthy position, then uses `vm.mockCall` to crash the USDC oracle price, pushing the health factor below 1. It then clears the mock with `vm.clearMockedCalls` (restoring real prices) and repays the borrowed amount. The final assertion confirms that the health factor is above 1, demonstrating that the position has been rescued from liquidation risk.

```
    function testFuzz_RepayRestoresHealthFactor(
        uint256 borrowAmt
    ) public {
        uint256 collateral = 10_000 * 10 ** USDC_DECIMALS;
        borrowAmt = bound(borrowAmt, 0.5 ether, 1 ether);
        deal(USDC, address(instance.account), collateral);
        supplyCollateral(collateral);
        borrow(borrowAmt);
        address oracle = IPoolAddressesProvider(ADDRESSES_PROVIDER)
            .getPriceOracle();
        address usdcSource = IAaveOracle(oracle).getSourceOfAsset(USDC);
        vm.mockCall(
            usdcSource,
            abi.encodeWithSignature("latestAnswer()"),
            abi.encode(int256(1))
        );
        (,,,,, uint256 healthBefore) = IPool(pool)
            .getUserAccountData(address(instance.account));
        assertLt(healthBefore, 1e18);
        vm.clearMockedCalls();
        repay(borrowAmt);
        (,,,,, uint256 healthAfter) = IPool(pool)
```

```
            .getUserAccountData(address(instance.account));
        assertGt(healthAfter, 1e18);
    }
```

Listing 5.15: Repay health factor restoration test

Together these tests confirm the full lifecycle of a borrowing position managed through the executor: supply collateral, borrow against it, and repay to close the debt and restore safety. The health factor restoration test is particularly instructive because it demonstrates a concrete use case for the repay action—recovering a position that would otherwise be liquidated.

6 Evaluation

This chapter evaluates the executor module from a quantitative angle. The most direct way to compare an ERC-7579 module against the way users currently interact with AAVE is to measure gas: how much does a typical workflow cost when issued through the module, and how does that compare with sending the equivalent transactions from a regular externally owned account (EOA)? Rather than building a dashboard that wraps the module in a user interface, I focus on the cost of the underlying on-chain operations, because that cost is the same regardless of which front-end is later layered on top.

The chapter first describes the methodology used to obtain the gas numbers, then presents the results across four representative workflows. It compares three configurations: an EOA calling AAVE directly, an ERC-7579 smart account issuing the same calls as a native batched UserOperation, and the same smart account issuing the workflow through my executor module. The chapter closes with a comparison against related solutions in the AAVE tooling ecosystem and a discussion of the limitations of the measurement.

6.1 Methodology

All measurements were produced by a Foundry test contract (`test/Gas.t.sol`) running against an Ethereum mainnet fork pinned at block 18,000,000. The fork block, EVM version (Cancun) and RPC endpoint are fixed in `foundry.toml` so the numbers are reproducible by re-running `forge test --match-contract GasTest -vv`. The compiler is run without optimizer overrides so the executor module is measured exactly as it is shipped in `src/`.

What is measured

For each workflow I record the total gas the user is responsible for paying. This includes the intrinsic transaction cost of 21,000 gas per on-chain transaction. The execution work (storage writes, oracle reads, AAVE bookkeeping) is the same in all three configurations; the cost difference comes from how many transactions are needed and from the overhead introduced by the ERC-4337 EntryPoint when a smart account is involved.

For the EOA configuration, each AAVE interaction is a separate transaction. I measure the pure execution gas of the calls with `gasleft()` snapshots and then add 21,000 for each transaction the user would have to broadcast. For the two smart account configurations, the entire workflow is bundled into

a single UserOperation routed through the EntryPoint via `handleOps`, and the bundler transaction itself pays the 21,000 intrinsic cost once.

Three configurations

EOA. The user calls AAVE directly. Supplying requires two transactions (`approve` followed by `Pool.supply`); borrowing requires one transaction; repaying requires another approval plus a `Pool.repay` transaction; withdrawing is a single transaction. This is the baseline that every comparison against AAVE-tooling gas costs must use, because it is the path most users take today.

Native ERC-7579 batch. The user owns an ERC-7579 smart account but does *not* install any application-specific module. They batch the same raw approve and Pool calls into one UserOperation using the account's native batched execution mode. This configuration isolates the cost of the account abstraction framework itself, separate from any cost that my module adds on top.

Smart account with the executor module. The same smart account, with the executor module from `src/ExecutorTemplate.sol` installed. Each user-level action (supply, borrow, repay, withdraw) is encoded as a single `executor.execute` call, and multi-action workflows are bundled as a batch of executor calls in one UserOperation. The module performs the necessary approvals internally, so the user's calldata only carries the high-level intent rather than the raw token-approval sequence.

Workflows

Four workflows of increasing length are measured:

1. **Supply** — supply 10,000 USDC to the AAVE pool. The EOA path needs two transactions because USDC must first be approved.
2. **Supply + borrow** — supply collateral, then borrow 0.5 WETH against it.
3. **Supply + borrow + repay** — the above, followed by an approval of WETH and a repayment of the same 0.5 WETH.
4. **Full cycle** — supply, borrow, repay, then withdraw half of the original collateral.

These cover the full lifecycle implemented in the module and let me observe how each configuration scales as more actions are added.

6.2 Results

Table 6.1 reports the total gas cost of each workflow under each configuration. All numbers are in raw gas units; the bundler base fee is already included for the smart account paths and the per-transaction base fee is already included for the EOA path.

Workflow	EOA	Native 7579	Module
Supply	251,824	502,911	525,475
Supply + borrow	510,996	760,165	798,728
Supply + borrow + repay	629,701	866,148	921,805
Full cycle (supply, borrow, repay, withdraw)	715,477	952,055	1,020,512

Table 6.1: Total gas cost per workflow, in gas units. EOA assumes one transaction per AAVE interaction; smart account configurations bundle the full workflow into a single UserOperation.

Smart account overhead is roughly constant

The gap between the EOA and the smart account paths sits in a narrow band: 274,000 gas for the shortest workflow, 305,000 for the longest. This is the overhead of running the same work through ERC-4337 rather than through a plain transaction: signature validation, nonce checks, EntryPoint accounting and the indirection between the EntryPoint, the account and the executor. Because this overhead is paid once per UserOperation, it is amortised across however many actions the workflow contains.

The break-even point follows directly from this. Each EOA transaction adds 21,000 gas in intrinsic cost. The smart account adds roughly 250,000 gas in framework overhead. A workflow would need to contain more than $\lceil 250{,}000/21{,}000 \rceil = 12$ transactions before the smart account becomes cheaper than the EOA. None of the four AAVE workflows I tested approach that threshold, and most realistic AAVE interactions a user would issue in practice do not either.

The module costs slightly more than a raw 7579 batch

Comparing the third column with the second isolates the cost of the executor module specifically. The module adds between 22,500 gas (single supply) and 68,500 gas (full cycle). This overhead comes from the additional contract hop introduced by `executor.execute`: the account calls into the module, the module decodes its action enum, and the module then calls back into the account using `_execute` to perform each underlying step. Each round trip and each ABI decoding adds a small but measurable cost.

The cost is not material on a per-workflow basis – under five percent for the shortest workflow, under eight percent for the longest. What the user gets in exchange is a smaller and safer surface to interact with: a single function with a typed action enum, a pool address and asset addresses fixed at install time, and approvals issued automatically rather than left for the user to manage. Whether that trade-off is worth paying for is a UX question, not a gas question.

The smart account path is not a gas-saving optimisation

The most important finding is that, for the workflows I measured, none of the smart account configurations are cheaper than the corresponding EOA workflow. This contradicts a common framing of account abstraction as a way to reduce on-chain costs by batching. Batching does eliminate per-transaction base fees, but only after enough operations have been batched to outweigh the fixed UserOperation overhead. For a single deposit-and-borrow that threshold is far out of reach.

The honest framing of this chapter, and of the project as a whole, is therefore that the module is not justified by gas. It is justified by the features that account abstraction enables and that the module exposes cleanly to those features: paymaster sponsorship so a user can interact with AAVE without holding ETH for fees, session keys so a delegated agent can execute pre-approved AAVE actions on the user's behalf, and a typed action interface that downstream tooling (validators, hooks, dashboards) can reason about without parsing raw approve and Pool calldata.

6.3 Comparison with other AAVE-tooling solutions

The executor module is one of several patterns developed to make interacting with AAVE more convenient than issuing raw transactions. This section places it in context against three families of existing solutions. None of these are benchmarked here in gas: they each rely on different account models and deployment assumptions, and a fair head-to-head benchmark would require re-deploying each of them at the same fork block. The comparison here is qualitative.

DeFi Saver and similar dashboards

DeFi Saver and other dashboards built on top of AAVE do not change the account model. The user still signs ordinary EOA transactions; the dashboard simply assembles convenient routes (for example, leveraged-position construction, automated liquidation protection) and prompts the user to approve them. The result is excellent UX for a single user but still requires multiple transactions and multiple signatures for any non-trivial flow, and it does not give a third party the ability to act on the user's behalf.

Instadapp DSA

Instadapp's Decentralised Smart Account (DSA) is closer in spirit to my module: it deploys a smart contract per user that owns positions across several DeFi protocols and exposes "connector" contracts which encapsulate protocol-specific logic. A user can ask the DSA to execute a sequence of connector calls in a single transaction. The architectural overlap with ERC-7579 is significant – both are smart-account-with-modules designs – but the DSA predates the ERC-4337 ecosystem, lives behind a custom permissioning model, and is not interoperable with bundlers or paymasters. The trade-off the DSA makes is the opposite of what my module makes: it gives up framework portability in exchange for being a self-contained ecosystem.

Safe modules

Safe (formerly Gnosis Safe) is the most widely deployed smart-account implementation, and modules can be installed on a Safe to extend its behaviour. AAVE-specific Safe modules exist in the wild and are conceptually similar to my executor: they receive a typed instruction and translate it into the underlying approve and Pool calls. The difference is the framework. Safe is not natively an ERC-4337 account and does not implement ERC-7579's standardised module interface. A module written for Safe is not portable to other accounts, whereas the module presented here implements `ERC7579ExecutorBase` and runs unchanged on any ERC-7579-conforming account (Kernel, Nexus, Safe7579, and so on). The portability is the main contribution of building on top of ERC-7579 rather than on top of any single smart-account implementation.

6.4 Limitations

The numbers reported in this chapter should be read with the following caveats in mind.

Single fork block. All measurements were taken at block 18,000,000. Gas costs for AAVE interactions can shift between blocks because of cold versus warm storage access and because AAVE upgrades can change the underlying logic. A campaign across many recent blocks would give a more robust picture but would not change the qualitative shape of the comparison.

Calldata cost. The 21,000 base fee added to each EOA transaction captures the intrinsic cost but not the per-byte calldata cost (4 gas per zero byte, 16 gas per non-zero byte). For `approve` and the AAVE Pool functions this adds roughly one to two thousand gas per transaction. The EOA numbers in Table 6.1 are therefore a slight under-count. Including calldata gas would make the EOA path marginally more expensive but would not affect the overall conclusion.

Bundler economics. The smart account numbers report the gas the EntryPoint charges. A real bundler typically marks up that figure with a priority premium and a paymaster surcharge if one is in use. These mark-ups are off-chain commercial choices and would not appear in any on-chain benchmark.

No paymaster, no session keys. The benchmark exercises only the plain user-signs-the-userOp path. The features that justify the smart account approach – gasless onboarding via paymasters, delegated signing via session keys – are not measured here because they shift the cost question from "how much gas?" to "who pays the gas, and what does the user have to hold?". That question is answered qualitatively in the preceding section.

Module configuration. The executor stores the pool address and two asset addresses at install time. The cost of installing the module is paid once per smart account and is not amortised in the per-workflow numbers above. For the test fixture, the install cost is roughly 25,700 gas (from the `testOnInstall` measurement in `test/ExecutorTemplate.t.sol`).

6.5 Discussion

The evaluation supports a single, honest conclusion: ERC-7579 plus an application-specific executor module is not the right tool for users whose goal is the lowest possible gas bill on a one-shot AAVE interaction. The EOA path remains the cheapest way to issue any short workflow against AAVE and is likely to remain so until the framework cost of an ERC-4337 UserOperation drops by an order of magnitude.

The tool is, however, the right shape for a different goal: giving users and the agents that act on their behalf a single, typed handle on AAVE that plays well with the rest of the account-abstraction stack. The module offers a stable interface (`execute(action, ...)`), encapsulates the correctness-critical sequencing of approve and Pool calls, and inherits all of the properties of the surrounding ERC-7579 ecosystem – programmable validators, hooks, session keys, and paymasters – without writing a single custom line for any of them. The 5–8% overhead the module adds on top of a raw batch is a fair price for that surface, and the 30–40% overhead the smart account path adds on top of the EOA path is the price one pays for the features themselves, not for the module.

7 Conclusion

A Appendix

Bibliography

[1] Aave. Aave v3 technical paper, 2022.

[2] Andreas M. Antonopoulos and Gavin Wood. *Mastering Ethereum: Building Smart Contracts and DApps*. O'Reilly Media, Sebastopol, CA, 2018.

[3] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. Erc-4337: Account abstraction using alt mempool, 2021.

[4] United States Bankruptcy Court for the District of Delaware. In re ftx trading ltd., case no. 22-11068, 2022. Chapter 11 bankruptcy filing, November 2022.

[5] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.

[6] zeroknots, Konrad Kopp, Taek Lee, Fil Makarov, Elim Poon, and Lyu Min. Erc-7579: Minimal modular smart accounts, 2023.